

AI 인프라실 기술 보고서

JAMYPG NL2SQL MCP 작동 원리 및 도구 상세 분석서

자연어 질의 인터페이스(MCP) 전송 계층 및 28종 도구 심층 설계 분석

작성 부서 AI 인프라실

발행 일자 2026년 7월 10일

문서 버전 v0.1.0

보안 구분 대외비 (Confidential)

보고 부서: AI 인프라실

보고 일자: 2026년 7월 8일

문서 버전: v1.1.0

비밀 등급: 대외비 (Confidential)

1. MCP (Model Context Protocol) 도입 배경 및 아키텍처 원리

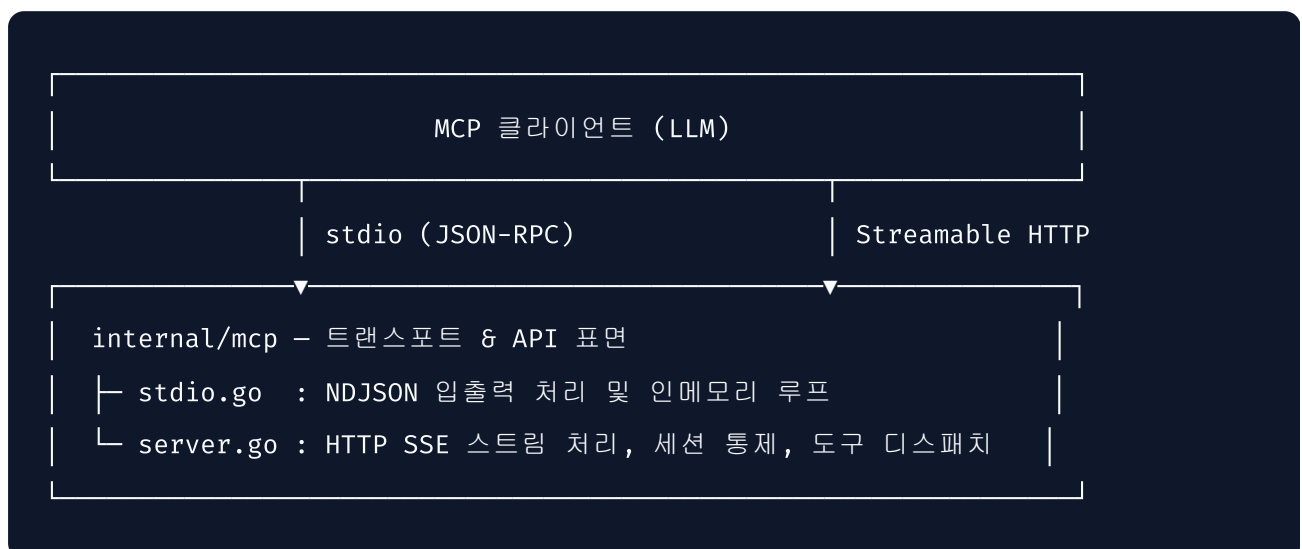
1.1. MCP 프로토콜 개요

자연어를 구조화된 SQL 질의로 변환하는 Text2SQL 환경에서 언어 모델(LLM)이 데이터베이스의 방대한 스키마 정보를 완벽히 파악하여 안전한 쿼리를 발행하기란 불가능에 가깝습니다. 데이터베이스 전체 정보를 LLM 프롬프트에 직접 주입할 시 **컨텍스트 오버플로**와 **비용 폭증**, 그리고 **할루시네이션(환각)**으로 인한 악성 쿼리 발행 및 **PII(개인식별정보)** 유출 위험이 도사리게 됩니다.

이에 AI 인프라실에서는 Anthropic 사가 주도하는 개방형 프로토콜인 **MCP(Model Context Protocol)**를 활용하여 금융 메타데이터 그라운드 아키텍처를 도입했습니다. LLM은 데이터베이스에 직접 질의하지 않고, JAMYPG MCP 서버가 정교하게 제어하는 추상화된 도구 계층(Tools)을 통과하며 단계적으로 근거를 수집 및 검증하는 구조를 갖추었습니다.

1.2. JSON-RPC 기반 전송 프로토콜 (Transports)

JAMYPG MCP 서버는 다음과 같은 두 가지 물리 전송 채널을 통해 표준 JSON-RPC 2.0 프로토콜을 통신합니다.



1) 표준 입출력 방식 (`stdio`)

- **작동 원리:** 클라이언트가 `jamypg-mcp` 바이너리를 서브프로세스로 기동한 뒤, 표준 입력(stdin)과 표준 출력(stdout) 채널을 뉴라인 구분자(`\n`)로 나뉘어진 JSON-RPC 2.0 형식(NDJSON)으로 양방향 통신합니다.
- **클라이언트 초기화 프로토콜 예시:**

```
{"jsonrpc":"2.0","id":1,"method":"initialize","params":{"protocolVersion":"2021-09-14"}}
```

- **보안적 의의:** 호스트 머신의 로컬 프로세스 경계 내에서 구동되므로 별도의 네트워크 개방이나 API 키 유포가 필요 없는 완전한 **로컬 신뢰 경계(Sandbox)**를 형성합니다.

2) 스트림 HTTP 방식 (`http`)

- **작동 원리:** 웹 리버스 프록시나 외부 게이트웨이 연동을 위해 HTTP 단일 엔드포인트 상에 Server-Sent Events(SSE) 스트림 채널을 개방합니다.
 - `POST /mcp` : 클라이언트에서 서버로의 JSON-RPC 요청을 전달하는 통로입니다.
 - `GET /mcp` : 서버에서 클라이언트로 비동기 응답이나 이벤트를 전달하는 SSE(text/event-stream) 전송 통로입니다.
 - `DELETE /mcp` : 할당된 세션 자원을 해제하고 즉각 커넥션을 안전하게 파기합니다.
- **세션 완대 정책 (Lenient Session Policy):** 서버는 클라이언트 호환성을 극대화하기 위해 `initialize` 요청 성공 시 발급되는 `Mcp-Session-Id` 헤더 전달을 의무화하지 않습니다. 헤더가 누락되거나 세션 관리가 미흡한 경량 호출자에 대해서도 무상태(Stateless) 형태로 정상 서비스를 보장하여 qwen-code, opencode 등 다양한 프레임워크와의 유연한 배선을 실현합니다.

1.3. 인증 및 네트워크 보안 통제 체계

바이너리가 HTTP 모드로 기동될 경우 내부망 외 노출에 대비하여 다층 보안 수단을 적용합니다.

- **Keycloak OIDC (SSO) 통합:** 조직 IdP(Identity Provider)와 표준 OIDC 흐름을 통합합니다. 웹 콘솔 (`/admin`) 로그인 시 Keycloak을 통해 발급받은 authorization code를 인계받아 `userinfo` API 를 거쳐 액세스 토큰의 서명 및 유효성을 백엔드에서 검증합니다.
- **API Key 관리 라이프사이클:** MCP 클라이언트 접근용으로 `jsk_` 접두사를 가지는 64자리 Hex API 키를 발급합니다. 데이터베이스(Postgres `jamypg_mcp_keys` 테이블)에는 키의 원본을 보관하지 않

고, **SHA-256 단방향 해시**만 보관하여 원본 탈취 사고를 차단합니다. 관리자는 만료일(TTL) 설정, 키 회전(Rotate) 및 즉각 폐기(Revoke) 권한을 가집니다.

- **Origin 및 CORS 검증:** 브라우저 교차 출처 스크립팅 방지를 위해 빈 Origin(비브라우저 스크립트), `localhost`, `127.0.0.1`, `::1` 및 실행 플래그나 설정으로 전달된 `-allow-origin` 허용 목록을 제외한 모든 외부 브라우저 호출을 `403 Forbidden` 으로 원천 거부합니다.

2. 내부 카탈로그 컴파일 및 핫스왑 메커니즘

JAMYPG는 실 데이터베이스에 우리가 가지 않도록 모든 메타데이터를 기동 시 단 한 번 **메모리 내 구조적 카탈로그로 컴파일**하여 연산합니다.

2.1. 13단계 컴파일 파이프라인

`catalog.Load(dataDir)` 메서드 호출 시 구동되는 로직은 다음과 같이 엄격한 검증 및 세밀한 추론 단계를 밟습니다.

1. **물리 모델 컴파일 (`loadPhysical`):** `meta_physical_models.json` 에서 물리 데이터베이스 테이블 및 컬럼 정의를 추출합니다. 파일 부재 또는 문법 파손 시 즉각 기동이 차단됩니다.
2. **논리 모델 컴파일 (`loadLogical`):** `meta_logical_models.json` 에서 물리 개체에 매핑되는 한국어 엔티티명과 속성명을 연결합니다. 물리 데이터 구조와 대조하여 적재되지 않거나 테이블 수가 0개이면 즉시 예외를 발생시키고 기동을 중단합니다.
3. **코드 사전 적재 (`loadCodeDict`):** `meta_code_dict.json` 에 기입된 공통 코드 명세를 로드합니다. `코드:라벨` 포맷을 분석하여 유효 리터럴 검사 및 값 매핑 사전의 뼈대를 형성합니다.
4. **조인 토폴로지 구성 (`loadRelations`):** `topology_relations.json` 에서 조인 그래프 엣지(Edge) 정의를 읽어 들여 인접 행렬(`c.Adjacency`)에 적재합니다.
5. **부가 메타데이터 로드:** 인덱스 정의(`loadIndexes`), 주제 영역 규칙(`loadSubjectAreas`), 데이터베이스 접속 정의(`loadDatabases`)를 로드합니다.
6. **업무 glossary 적재:** `glossary.json` 에 정의된 핵심 엔티티/메트릭 동의어 사전을 메모리에 파싱합니다.
7. **지표 사전 적재 (`loadMetrics`):** `metrics.json` 에서 표준 지표 계산 공식(`expression`), 필수 정책 필터 등을 로드합니다.
8. **Dialect 바인딩:** `databases.json` 의 `dbms` 지정값(POSTGRES/MYSQL/MARIADB) 또는 `overrides.json` 의 `dialect` 항목에 근거하여 `postgres` (기본), `mysql`, `mariadb` 중 SQL

번안 방안을 설정합니다.

9. **Semantic Type 추론**: 물리 컬럼의 명칭(예: `..._DT`, `..._YM`), 데이터 타입 및 길이를 기반으로 `DATE`, `TIMESTAMP`, `DATE_YYYYMMDD`, `MONTH_YYYYMM` 등의 시계열 속성을 자체 알고리즘에 의해 자동 판정합니다.
10. **Overrides 패치**: `overrides.json` 을 읽어 들여 특정 컬럼의 설명 보정, PII 보안 컬럼 지정, 금지된 조인 관계 조합, 기본 정책 필터(`status_code IS NULL` 등)를 물리 모델 정보 위에 덮어씁니다.
11. **개인정보 보호(PII) 속성 바인딩**: `overrides.json` 에 열거된 PII 컬럼들과 `*.ssn_hash` 와 같은 와 일드카드 패턴을 종합하여 해당 컬럼의 PII 플래그를 `true` 로 설정하고 검색 단계에서 선제 차단할 준비를 마칩니다.
12. **컬럼 프로파일 통계 바인딩**: `column_stats.json` 이 존재할 경우 Null 비율, 고유값(Distinct) 개수, 데이터 분포 Top-N 리스트를 컬럼 객체에 바인딩하여 최신성과 데이터 규모를 인덱싱합니다.
13. **상호 참조(Cross-Reference) 유효성 검사**:
 - 관계 토폴로지 내 조인 컬럼이 물리 스키마 테이블에 실제로 존재하는지 대조합니다.
 - 지표 정의에 나열된 테이블과 컬럼들이 실제 카탈로그에 실존하는지 상호 검증합니다.
 - 불일치 사항은 차단하지 않고 `LoadIssue` 경고 목록으로 수집하여 모니터링 콘솔에 노출합니다.

2.2. atomic.Pointer 기반 핫스왑 및 트랜잭션 롤백

JAMYPG는 메타데이터가 운영 중에 유동적으로 수정되는 환경에서 고성능을 내기 위해 인메모리 컴파일 객체를 단일 포인터로 제어합니다.

```
type Server struct {
    catalogPtr atomic.Pointer[catalog.Catalog] // 활성 인스턴스 (락 프리 읽기)
    dataMu      sync.Mutex                       // 쓰기 작업 상호 배제 뮤텍스
}
```

- **락 프리 읽기**: 도구 호출이나 질의 엔진은 `s.catalogPtr.Load()` 를 통해 활성화된 카탈로그 스냅샷을 원자적으로 로드하여 연산합니다. 이 과정에서 뮤텍스 락이 걸리지 않아 극도의 초고속 처리가 가능합니다.
- **가상 격리 빌드 및 롤백**: 데이터셋 수정이 발생하면 `dataMu.Lock()` 하에 격리된 임시 디렉터리에서 카탈로그를 통째로 다시 빌드합니다. 컴파일 결과 에러가 단 한 건이라도 발견되거나 신규 `LoadIssue` 가 임계치를 넘으면 컴파일을 기각하고 파일을 이전 백업으로 자동 복원(롤백)합니다.

- **원자적 교체**: 신규 컴파일이 무결하게 완료되면 `s.catalogPtr.Store(newCatalog)` 를 통해 메모리 주소를 원자적으로 전환합니다. 교체 이전에 진입한 세션은 이전 카탈로그 주소를 들고 프로세스가 끝날 때까지 처리되므로 트래픽 정체는 Zero화됩니다.

3. 자연어-SQL 변환 도구 오케스트레이션 설계

JAMYPG는 프롬프트에 스키마 전체를 전달하지 않고, 정해진 10단계 표준 연동 파이프라인을 거치며 최소한의 "사실(Facts)"에만 입각하여 쿼리를 완성하도록 에이전트를 유도합니다.

3.1. GraphRAG 기반 검색 확장 계층 (Retrieve Context)

JAMYPG v0.12.0부터 단순 키워드/시드 검색의 한계를 극복하기 위해 **GraphRAG 기반 컨텍스트 검색 기법**이 도입되었습니다.

- **시드 검색 (Seed Search)**: 1차 질문 텍스트 분석을 바탕으로 높은 유사성을 보이는 핵심 테이블군을 추출합니다.
- **1-hop 조인 확장**: 1차 추출된 고신뢰성 시드 테이블을 거점으로 조인 토폴로지 상의 1-hop 이웃 관계에 놓인 테이블들을 자동 탐색에 포함시킵니다.
- **하이브리드 재정렬 (Re-ranking)**: 아래에 설명할 7개 가중치 신호 결합 수식에 의해 최종 테이블 랭킹 및 컬럼 증거를 통합 재산출하여 컨텍스트에 융합(Fusion)합니다.

3.2. 되묻기(Clarification) 판단 기준 및 환류 학습 루프

질문의 정보가 부족하거나 모델 오작동 위험이 높을 경우, 서버는 질문 분해 단계에서 실행을 중단하고 `needs_clarification` 상태와 구체적인 선택지를 반환합니다.

- **Blocking (중단 조건)**:
 - **애매한 질문 (`too_vague`)**: 질문의 자모 길이가 10자 미만이며 시간이나 범위 구절이 없어 분석 영역 지정이 불가능한 경우.
 - **미지정 지표 (`metric:kw`)**: 질문에 "연체율" 등의 키워드가 포함되었으나 지표 사전에 정의가 없는 경우 (추정 지표 리스트를 Option으로 나열).
 - **테이블 근접 점수 경쟁 (`table_choice`)**: 테이블 매칭 점수가 최상위권에서 박빙(Score 차이 8% 이내)이면서 두 테이블의 소속 도메인이나 집계 단위(Grain)가 달라 통계 왜곡 위험이 큰 경우.

- **필터 컬럼 모호성 (`column_choice`)**: "서울"과 같은 리터럴이 2개 이상의 컬럼과 동등한 스코어로 매핑되어 Where 조건을 특정할 수 없는 경우.
- **재질문 학습 루프 (v0.14.0)**: 사용자가 제공된 Option(예: `a` 또는 `b`)을 클릭하여 응답을 주어 `status: "ready"` 로 해소되면, 서버는 해당 선택 테이블을 **`outcome: "corrected"` , `source: "clarification"` 형태의 피드백 데이터로 즉시 자동 기록합니다.** 다음 카탈로그 리로드 시 이 피드백 이력이 검색 가점(`FeedbackUsage` 및 `usage` prior)으로 자동 편입되어, 차후 유사한 질문 발생 시 사용자가 과거 확정했던 테이블이 발견 순위에서 영구적으로 우대됩니다.

4. 28종 MCP 전체 도구 상세 분석

JAMYPG 서버가 배선하는 **28종 전체 도구**의 물리 파라미터 규격, 상세 소스 내부 동작 알고리즘 및 비즈니스 목적을 단계별로 전수 분석합니다.

Phase 1: 자연어 질문 이해 및 해석 (질문 분석 및 어휘 변환)

1) `prepare_sql_context`

- **입력 규격 (Parameters)**:
 - `question` (string, 필수): 자연어 질문.
 - `tables` (array, 선택): 우선 선택할 스키마 한정 테이블 목록.
 - `limit` (integer, 선택): 기본 1000행 제한.
 - `clarifications` (object, 선택): 이전 턴에 제기된 되묻기 응답 맵.
 - `previous_question` (string, 선택): 대화형 후속 턴 처리를 위한 이전 질문 내용.
 - `previous_sql` (string, 선택): 이전 턴에서 산출되었던 SQL 구문.
- **내부 동작 알고리즘 (`prepare.go`)**:
 - `ResolveClarifications` 를 가동해 사용자의 되묻기 응답이 있다면 이를 working 질문 텍스트로 치환/병합합니다.
 - `DetectClarifications` 를 수행해 blocking 레벨의 불확실성이 검출될 시, 스켈레톤 조립을 보류하고 `status: "needs_clarification"` 과 함께 즉각 실행을 중단합니다.

- 모호성이 해결되면 내부적으로 **GraphRAG 검색기(RetrieveContext)**를 가동하여 시드 검색에 조인 1-hop 확장을 더한 하이브리드 테이블 랭킹을 산출합니다.
- 질문 속 어휘를 기반으로 지표 정의 조회(LookupMetrics), 컬럼 프로파일 검색, 조인 토폴로지 분석(GetJoinPaths)을 차례로 통합 가동합니다.
- 최종 뼈대인 BuildSQLSkeleton 을 빌드해 하나의 통합 컨텍스트 JSON 번들로 압축 반환합니다.
- **재질문 환류 피드백**: 사용자가 답변을 제공해 status: "ready" 가 완료되면 선택 테이블을 outcome: "corrected" , source: "clarification" 피드백으로 자동 기록합니다.
- **1회 응답 보장 정책(One-round guarantee)**: 만약 사용자가 clarifications 에 값을 제공한 두 번째 요청의 경우, 새로 Surfaced된 모호함은 advisory 수준으로 강하게 무한 질문 루프에 빠지는 현상을 차단합니다.
- **비즈니스 목적**: 단 한 번의 호출로 자연어 해석부터 골격 생성까지 원스톱으로 처리하여 LLM이 검증 단계를 생략하고 독단적으로 SQL을 날조하는 행위를 원천 방지합니다.

2) analyze_question

- **입력 규격 (Parameters)**: question (string, 필수)
- **내부 동작 알고리즘 (analyze.go)**:
 - 한국어/영어 자연어 형태소와 업무 사전을 대조하여 질문의 의도(Intent)와 시그니처 (agg_sum_groupby_sort 등)를 추정합니다.
 - 질문 내 명시된 기대 차원(Dimension), 적용되어야 할 지표 리스트, 정렬 조건, 상위 N개 한계 등을 분석하여 expected_output_columns 구조로 정형화해 출력합니다.
- **비즈니스 목적**: 텍스트에 포함된 수치 제한, 시간 범위, 집계 단위를 분해하여 기계적인 검증 메타데이터로 사전에 확정해 둡니다.

3) resolve_time

- **입력 규격 (Parameters)**: question (string, 필수), table (string, 선택)
- **내부 동작 알고리즘 (timeparse.go)**:
 - "어제", "지난 분기", "최근 6개월" 등의 시계열 키워드를 정적 달력 매핑 테이블과 결합해 절대 날짜 범위(Start~End)로 전환합니다.
 - table 명이 명시된 경우, 카탈로그 내 해당 테이블의 컬럼 중 날짜/시간 정보를 지닌 타겟 컬럼의 물리 데이터타입(DATE_YYYYMMDD , MONTH_YYYYMM 등) 스펙에 맞추어 YYYYMMDD 혹은 YYYYMM 형식의 완벽한 WHERE 조건문 문자열(예: T1.base_month = '202506')을 빌드합니다.

- **비즈니스 목적:** 언어 모델이 DB별 날짜 형식 변환 함수(`TO_DATE` , `STR_TO_DATE` 등)나 기준 포맷을 몰라 에러를 내거나 잘못된 날짜 컬럼을 지정하는 행위를 근절합니다.

4) `find_filter_columns`

- **입력 규격 (Parameters):** `values` (array, 필수), `tables` (array, 선택), `top_k` (integer, 선택)
- **내부 동작 알고리즘 (`stats.go`):**
 - 질문 속 어휘("정상", "개인사업자" 등)를 코드 사전에 등록된 라벨명, 프로파일 통계의 최빈값(Top Values) 데이터와 텍스트 비교 대조합니다.
 - 매칭 신뢰도 스코어를 계산하여 가장 가능성이 높은 컬럼명과 타겟 값 리터럴을 매핑(예: `cust_type = '01'`)하여 SQL 조건 술어 형태로 변환해 제공합니다.
- **비즈니스 목적:** 사용자가 한국어 명칭으로 물어본 상세 공통 코드 값을 데이터베이스 내의 실제 코드값 (영문/숫자 코드)으로 매칭시키는 번역기 역할을 담당합니다.

Phase 2: 스키마 탐색 및 통계 분석 (테이블 및 컬럼 위치 판정)

5) `retrieve_context` [신규]

- **입력 규격 (Parameters):** `question` (string, 필수), `top_k` (integer, 선택)
- **내부 동작 알고리즘 (`graph.go`):**
 - **시드 검색 및 순위 보존 융합:** 먼저 `SearchSchema` 를 기동하여 연관성이 높은 시드(Seed) 테이블들을 추출합니다.
 - **1-hop 조인 인접 확장:** 신뢰도 스코어가 높은 시드 테이블들을 거점으로 이들과 조인 릴레이션이 직접 성립되는 이웃(Neighbor) 테이블들을 그래프 인접 행렬(`c.Adjacency`)에서 1-hop 스캔하여 풀에 추가합니다.
 - **값 증거 탐색:** 질문에 등장한 리터럴 텍스트를 `FindFilterColumns` 에 통과시켜 값 일치 증거가 감지된 테이블도 풀에 병합합니다.
 - **7-신호 하이브리드 스코어 계산:** 후보 테이블에 대해 다음 수식에 따라 점수를 누적 연산합니다.

$$\text{Score} = w_{\text{Semantic}} S_{\text{Semantic}} + w_{\text{Lexical}} S_{\text{Lexical}} + w_{\text{Proximity}} S_{\text{Proximity}} + w_{\text{Join}} S_{\text{Join}} + w_{\text{Value}} S_{\text{Value}} + w_{\text{Usage}} S_{\text{Usage}} + w_{\text{Freshness}} S_{\text{Freshness}}$$

- 가중치 비중: `Semantic` = 0.55 (dominant), `Lexical` = 0.10, `Value` = 0.10, `Joinability` = 0.09, `Proximity` = 0.08, `Usage` = 0.04, `Freshness` = 0.04.

- **발견 증강 추가 (Discovery Append):** 순위 보존 융합을 위해 시드 검색 결과들의 순서는 그대로 유지하되, 시드 풀에 포함되지 않았으나 상기 하이브리드 스코어 연산에서 최고점을 득한 비-시드 발견 후보 테이블을 최대 2개까지 리스트 테일에 덧붙여(`append`) 반환합니다.

- **비즈니스 목적:** 키워드 단독 검색으로 놓치기 쉬운 조인 대상 배후 테이블이나 데이터 값 증거 테이블을 GraphRAG 아키텍처를 기반으로 완벽 복원하여 검색 정확도를 극대화합니다.

6) `search_schema`

- **입력 규격 (Parameters):** `question` (필수), `top_k` (선택), `schemas` (선택), `include_columns` (선택), `max_columns` (선택)
- **내부 동작 알고리즘 (`search.go`):**
 - 질문 어휘가 테이블 물리명, 논리명, 설명, glossary 동의어에 출현할 시 1차 렉시컬 매칭 점수를 산출합니다.
 - 컬럼 가점(상위 매치 컬럼들의 개별 점수 합산)과 지표 가점(지표 사전 정확 일치 시 flat +12점), 과거 성공 피드백 가중치를 대조하여 랭킹 점수를 도출합니다.
 - 1차 매칭 완료 후 상위 Top 풀(`TopK * 2` 이내) 내에서 서로 조인이 성립되는 관계가 토폴로지 상에 존재하는 경우, isolated(고립) 테이블을 방어하기 위해 **조인 연결 가점(+4점)**을 부여한 뒤 최종 랭킹을 다시 정렬합니다.
 - `TopK` 밖으로 밀려 탈락한 하위 후보 테이블들에 대해 `ExcludedCandidate` 리스트에 수집하고, 탈락 사유(매칭된 약한 신호 목록)를 기재해 반환합니다.
- **비즈니스 목적:** 수백, 수천 개의 사내 테이블 중 질문 해결에 가장 적합한 실시간 타겟 테이블 세트를 안 전하고 정밀하게 도출합니다.

7) `search_examples`

- **입력 규격 (Parameters):** `question` (string, 필수), `top_k` (integer, 선택), `table` (string, 선택)
- **내부 동작 알고리즘 (`search.go`):**
 - `sql_datasets.json` 에 저장된 실사례 질문 데이터셋을 렉시컬(Lexical) 점수로 검색합니다.
 - 질문의 어휘 매치 점수에 더해 `analyze_question` 이 뽑은 문장 구조 인텐트 시그니처가 정확히 겹치는 항목에 대해 **토큰당 2점의 가점**을 부여해 동일 구조의 쿼리를 Few-shot 후보로 우선 추출합니다.

- **비즈니스 목적:** 과거 사람이 튜닝하여 적재한 성공 Golden SQL 쿼리 중 가장 유사한 문법 구조를 갖춘 예제를 우선 제시해 LLM이 패턴을 학습하도록 돕습니다.

8) `get_column_stats`

- **입력 규격 (Parameters):** `table` (string, 필수), `column` (string, 필수)
- **내부 동작 알고리즘 (`stats.go`):**
 - 지정한 테이블과 컬럼에 대해 카탈로그 내 등록 메타데이터와 프로파일 통계 (`column_stats.json`)를 취합합니다.
 - 컬럼의 물리 유형, PK/FK 속성, PII 컬럼 해당 여부, Null 비율, Distinct Count, 샘플값 분포 및 골든셋 쿼리에서의 사용 횟수를 리턴합니다.
- **비즈니스 목적:** 특정 컬럼의 실데이터 통계를 LLM에 참조시켜 해당 속성이 필터 연산이나 집계 처리에 적합한 컬럼인지 판단하게 만듭니다.

Phase 3: 업무 도메인 근거 수집 (비즈니스 지표 및 관계 탐색)

9) `get_metric_definition`

- **입력 규격 (Parameters):** `metric_name` (string, 필수), `top_k` (integer, 선택)
- **내부 동작 알고리즘 (`metrics.go`):**
 - `metrics.json` 원장에서 요청 지표의 이름 및 별칭(Aliases)과 한국어 매치를 시도합니다.
 - 매칭 성공 시 `source: "dictionary"` 유형으로 확정 계산식(`Expression`), 필수 필터 구문, 분석 제외 가이드를 제공합니다.
 - 실패 시 물리 컬럼 명명 패턴을 기반으로 기계적인 집계 방식(Sum, Count 등)을 역산하여 `source: "inferred"` 후보군으로 제안합니다.
- **비즈니스 목적:** 비즈니스 지표 계산식을 LLM이 자기 멋대로 오해하여 잘못된 집계 공식을 발행하지 못하도록 지표 사전을 강제 배선했습니다.

10) `get_join_paths`

- **입력 규격 (Parameters):** `from_table` (선택), `to_tables` (선택), `tables` (선택), `max_depth` (선택)
- **내부 동작 알고리즘 (`join.go`):**

- 관계 토폴로지 조인 그래프에서 BFS(너비 우선 탐색)를 구동하여 테이블 간 최단 연결 경로와 물리 매핑 키 컬럼을 식별합니다.
- 수동 지정 엣지(Operator/Manual 지정은 가중치 0.95, 일반 추정은 0.6) 기반의 연결성 신뢰도 점수를 합산하여 최종 경로의 `confidence` 를 출력하며, 만약 `confidence < 0.7` 미만이거나 단절이 있을 경우 저신뢰 안내 문구(`guidance`)를 작성합니다.
- `overrides.json` 에 기입된 **금지 조인 조합** 탐지 시 즉시 에러와 사유를 반환하고 경로를 기각합니다.
- **비즈니스 목적:** 쿼리 폭주(Cartesian Product) 및 잘못된 속성 키 연결로 인한 조인 연산 에러와 통계 훼손을 완벽히 도단합니다.

11) `get_schema_context`

- **입력 규격 (Parameters):** `question` (선택), `tables` (array, 선택), `max_columns_per_table` (선택)
- **내부 동작 알고리즘 (`search.go`):**
 - 넘겨진 테이블들에 대해 LLM이 해독하기 가장 알맞은 메타데이터(타입, PK/FK 여부, PII 여부, 기본 필터 정책 등)만을 골라내고 불필요한 속성 컬럼을 잘라내어 **컨텍스트를 최소 크기로 압축**합니다.
 - 제외된 컬럼 정보와 사유는 `excluded_columns` 이력 구조에 담아 투명성을 제공합니다.
- **비즈니스 목적:** LLM의 프롬프트 토큰 사용 효율성을 대폭 개선하고 주의력 분산으로 인한 컬럼 오지정 환각 오류를 억제합니다.

Phase 4: SQL 구조화 조립 및 검증 (스켈레톤 조립 및 정적 가드레일)

12) `build_sql_skeleton`

- **입력 규격 (Parameters):** `question` (필수), `tables` (선택), `limit` (선택)
- **내부 동작 알고리즘 (`skeleton.go`):**
 - 지정된 테이블 목록 순서에 맞게 고유의 테이블 앨리어스(`T1` , `T2` 등)를 원자적으로 할당하고, 관계 엣지 조건 내의 테이블 식별자를 앨리어스로 물리 치환하여 JOIN 구문을 결합 조립합니다.
 - 질문 내용에 맞는 날짜 컬럼을 Preferred 우선 순위에 따라 탐지하여 기간 필터 조건을 WHERE 절에 자동 장착합니다.

- overrides에 등록된 기본 필터(`status_code IS NULL` , `COALESCE(is_active, 'Y')` < 'N' 등)와 Point-in-time 시점 필터를 조건절에 선제 탑재하고, LLM이 채워야 하는 가변적 필터나 SELECT 리스트 영역을 `/* SLOT */` 주석으로 마킹한 뒤 말미에 `LIMIT` 행 제한을 부착해 출력합니다.
- **비즈니스 목적:** 금융권 SQL에 필수로 들어가는 시점/상태 필터링과 복잡한 조인 구문의 완성 권한을 서버가 담당하여 쿼리 골격의 기초 무결성을 확보합니다.

13) `validate_sql`

- **입력 규격 (Parameters):** `sql` (필수), `limit` (선택), `metrics` (선택), `expected_outputs` (선택)
- **내부 동작 알고리즘 (`validate.go`):**
 - 문장 내 리터럴 문자열과 주석을 `maskSQL` 을 가동해 무해한 단어로 치환(Masking)한 뒤 AST-like 정적 토큰 분석을 전개합니다.
 - SELECT/WITH 조회 전용 여부(`NOT_SELECT`), 위험 키워드 포함 여부(`BLOCKED_KEYWORD`), 방언 위반 여부를 체크합니다. Oracle 전용 잔존 구문(`NVL` , `DECODE` , `ROWNUM` , `ADD_MONTHS` , `LISTAGG` 등)은 `DIALECT_FUNCTION` 에러로 기각하고, MySQL/MariaDB 방언에서의 `FETCH FIRST` 사용은 `DIALECT_LIMIT` 에러로 차단하며(`LIMIT n` 치환 안내), 교차 방언 함수는 경고로 표출합니다.
 - 추출된 사용 테이블/컬럼명이 카탈로그 원장에 존재하는지 교차 대조하고 개인정보 보호 컬럼 검사(`PII_COLUMN`), 비인가 조인 검사(`FORBIDDEN_JOIN`), 카티션 조인 검사(`JOIN_WITHOUT_ON`), 필수 정책 필터 누락 검사, 코드값 유효성 검사(`CODE_VALUE_UNKNOWN`) 등 **33종의 룰셋을 전수** 가 통합합니다.
 - 실패 시 에러 사항을 기계적으로 자동 치환할 수 있도록 `fix_hints` 패키지를 구성하여 반환합니다 (클라이언트 루프 2회 통제용).
- **비즈니스 목적:** 작성 완료된 최종 SQL 쿼리가 내부 거버넌스와 보안 정책, 데이터 스펙에 완벽히 부합하는지 실행 전에 실시간 정적 통제합니다.

14) `rank_candidates`

- **입력 규격 (Parameters):** `question` (선택), `candidates` (array, 필수), `expected_outputs` (선택), `metrics` (선택), `limit` (선택)
- **내부 동작 알고리즘 (`rank.go`):**
 - 입력된 복수의 후보 SQL들에 대해 일괄적으로 `ValidateSQL` 검사 및 `ExplainSQL` 정적 리스크 분석을 구동합니다.

- 내부 감점 공식에 입각하여 점수를 집계합니다 (기본 100점 시작, 검증 통과 실패 시 `-1000점` 각각, 룰 에러당 `-50점`, 경고 유형별 차등 감점 `-25~-5점`, 리스크 스코어 감점 등).
- 가장 무결하고 위험 점수가 낮은 최적의 쿼리를 `best_sql` 로 판정하여 재배열합니다.
- **비즈니스 목적:** LLM의 주관적이고 비결정적인 쿼리 자가 평가 방식을 배제하고, 서버 통제 규칙에 입각해 가장 정교하고 안전한 최선책을 선택합니다.

Phase 5: 실행 계획 분석 및 안전 실행 (실측 리스크 평가 및 실행 제어)

15) `explain_sql`

- **입력 규격 (Parameters):** `sql` (string, 필수), `limit` (integer, 선택), `profile` (string, 선택)
- **내부 동작 알고리즘 (`validate.go`, `dbconn` 커넥터):**
 - 쿼리 내 WHERE 절 누락 여부 등 정적 요소들로 1차 Risk 스코어를 계산합니다.
 - `profile` 접속 ID가 지정된 경우, 실제 대상 DB에 방언별 JSON EXPLAIN을 날립니다.
PostgreSQL은 `EXPLAIN (FORMAT JSON) [SQL]`, MySQL/MariaDB는 `EXPLAIN FORMAT=JSON [SQL]` 을 실행하여 반환된 물리 플랜 JSON 트리를 직접 로드해 2차 분석합니다.
 - 플랜 스텝 중 Full Scan 비용, Cartesian Product 의심 조인(조인버퍼 조인 포함), 대형 정렬, 예상 Cost 등을 스캔하여 위험도 등급(`risk: low|medium|high`)을 확정합니다. 실측 결과가 high인 경우 실행 차단을 강제 권고합니다.
- **비즈니스 목적:** 작성된 쿼리가 인프라 수준에서 데이터베이스 자원을 폭식하거나 타임아웃 장애를 유발할 쿼리인지 미리 실측 진단하여 방어합니다.

16) `list_db_profiles`

- **입력 규격 (Parameters):** 없음 (권한 필터 적용)
- **내부 동작 알고리즘 (`dbapi.go`, `auth.go`):**
 - 호출 계정의 역할(Role)과 개별 Visibility(소유 권한, 공유 설정인 `shared` 여부, 타인에게 `grant` 수여된 프로파일 권한) 정보를 Postgres 권한 맵에서 확인합니다.
 - 권한이 유효한 PostgreSQL/MySQL/MariaDB 접속 프로파일들의 ID와 상세 접속 사양 정보 (`type`, `connect_string` 등)를 정제하여 반환합니다 (비밀번호 참조 `password_ref` 등은 마스킹하여 보안 노출 차단).

- **비즈니스 목적:** 호출 클라이언트와 타깃 LLM 에이전트가 본인의 권한 범위 내에서만 적절한 타깃 DB 프로파일 식별자를 식별 및 지정할 수 있도록 접근 지도를 제공합니다.

17) `run_sql_safely`

- **입력 규격 (Parameters):** `sql` (필수), `profile` (선택), `limit` (선택), `timeout_seconds` (선택), `fresh` (선택)
- **내부 동작 알고리즘 (`execguard.go` , `dbconn` 커넥터):**

 - **세션 락 검사:** 본 요청 세션 내에 아직 해결되지 않은 Blocking 레벨의 되묻기 질문 이력이 존재하고 실 DB 접속을 시도하는 경우, 실행을 거부하고 `status: "clarification_required"` 를 즉각 호출자에게 반환합니다.
 - **검증 게이트:** `ValidateSQL` 결과 에러가 존재하면 DB 쿼리를 전혀 수행하지 않고 즉각 실행 차단 (`status: "blocked"`) 처리합니다.
 - **행수 제한 강제화:** 조회 쿼리를 `SELECT * FROM ( [SQL] ) AS jampg_q LIMIT maxRows+1` 형태로 감싸 대상 DB 메모리 오버헤드를 제어합니다 (MySQL/MariaDB의 WITH 구문 쿼리는 파생 테이블 래핑 대신 말미에 `LIMIT` 을 직접 부착).
 - **결과 캐시 및 PII 마스킹:** 60초 결과 캐시(`cached: true`)를 활용해 무분별한 반복 조회 부하를 막고, 설정된 PII 컬럼이 최종 조회 결과 데이터 셋에 출현할 시 값을 마스킹 필터링 처리합니다.
 - **서킷 브레이커 가동:** 대상 프로파일에 대해 연속 3회 쿼리 에러 또는 접속 타임아웃 발생 시, 서킷 브레이커 상태를 `OPEN` 으로 전환해 30초간 모든 접속 행위를 현관에서 즉시 기각합니다.

- **비즈니스 목적:** 보안 컴플라이언스를 만족하고 실 데이터베이스 서버를 보호하기 위한 최종 read-only 쿼리 런타임 가드레일을 담당합니다 (DSN 수준 read-only 세션 강제와 함께 다층 방어를 구성).

Phase 6: 피드백 적재 및 반복 학습 (자동 환류 학습)

18) `record_feedback`

- **입력 규격 (Parameters):** `question` (필수), `outcome` (필수), `generated_sql` (선택), `final_sql` (선택) 등 피드백 메타데이터
- **내부 동작 알고리즘 (`feedback.go`):**

 - 호출 결과물(질문, 변환 SQL, 검증 에러 목록, 사용자 교정본, 최종 채택 여부인 `adopted` 등)을 수집하여 디스크 내의 일자별 `feedback/feedback-YYYYMMDD.jsonl` 파일에 순차 적재합니다.

- **Fewshot 피드백 연계:** 채택에 성공한 쿼리 레코드는 서버 재기동/리로드 시 자동 메모리 Few-shot 데이터셋과 `search_schema` 빈도 가점에 자동 연계됩니다.
- **비즈니스 목적:** NL2SQL 실서비스 구동 실적 데이터와 사용자 교정 사례를 데이터셋에 축적하여 정확도 개선 피드백을 확보합니다.

19) `learn_from_feedback`

- **입력 규격 (Parameters):** `min_occurrences` (integer, 선택)
- **내부 동작 알고리즘 (`learn.go`):**
 - 축적된 `feedback/*.jsonl` 로그와 실 DB 실행 쿼리 감사 로그(`db:execute` 항목)를 분석하여 반복 에러가 터지는 지점을 탐색합니다.
 - 동일 테이블에서 PG-/MY- 계열 실행 에러나 타임아웃이 \$N\$회 이상 터질 시 `recurring_exec_error` 로, 특정 컬럼이 다른 컬럼으로 반복 교체된 경우 `column_correction` 으로, 조회가 느린(≥ 5 초) 테이블인 경우 `slow_query` 룰로 식별합니다.
 - 도출된 패턴을 `learned_rules.json` 파일에 영속화 및 메모리에 즉각 장착하여, 이후 해당 테이블 사용 시 `ValidateSQL` 에서 경고 힌트(`LEARNED_SLOW_QUERY` 등)를 사전 표출하게 합니다.
- **비즈니스 목적:** 시스템 스스로 대상 데이터베이스 성능 병목과 에이전트 반복 실수 패턴을 상시 모니터링하여 가이드라인에 능동 환류하는 자동 튜닝 루프를 제공합니다.

Phase 7: 서버 라이프사이클 및 데이터셋 운영 (인프라 제어 및 메타데이터 관리)

20) `suggest_join_relations` [신규]

- **입력 규격 (Parameters):** `golden_path` (string, 선택)
- **내부 동작 알고리즘 (`joingap.go`):**
 - 지정한 골든셋 쿼리 파일(`golden_queries.json`)을 분석하여 기대 테이블 간에 조인 릴레이션이 성립하지 않아 조인 경로 검색율(Join Path Recall)을 깎아 먹는 '**조인 단절 구간(Join Path Recall Gap)**'을 전수 탐지합니다.
 - 탐지된 단절 쌍 테이블들에 대해 공통된 식별자 키 성격을 가지는 컬럼들(`_NO` / `_ID` 접미사 우선, 차순위 `_CD` / `_SNO` / `_KEY`)을 식별하여 병합 키 후보군을 랭킹 산출합니다.
 - 운영자가 바로 복사해 붙여넣을 수 있도록 무결한 `relations.json` 설정 규격 스니펫(`suggested_relation`)을 제공합니다.

- **비즈니스 목적:** 골든셋 정확도 회귀를 유발하는 실제 관계 정보 유실 지점을 자동으로 찾아내어 카탈로그 지식을 완성할 수 있는 가이드를 생성합니다.

21) `get_catalog_health`

- **입력 규격 (Parameters):** 없음
- **내부 동작 알고리즘 (`health.go`):**
 - 활성 카탈로그 메모리 상태를 진단하여 `ok` / `degraded` (컴파일 경고 존재) / `error` (치명적 물리 구조 붕괴) 여부를 출력합니다.
 - 교차 유효성 갭 검사에서 집계된 이슈 전체 목록, 논리명 누락 필드 목록, PII 보호 컬럼 전체 리스트 및 사전에 정의된 데이터 건수 지표를 취합해 요약 보고합니다.
- **비즈니스 목적:** 카탈로그 메타데이터의 컴파일 품질과 안전 상태를 상시 진단하는 헬스 모니터링 원장입니다.

22) `run_evaluation`

- **입력 규격 (Parameters):** `golden_path` (선택), `top_k` (선택), `profile` (선택), `retrieval` (boolean, 선택)
- **내부 동작 알고리즘 (`eval.go` , `eval_retrieval.go`):**
 - 지정한 골든셋 쿼리 파일에서 질문 목록을 순회 검사하며 테이블 매칭률, 컬럼 재현율, 조인 경로 정합성 점수를 수학적으로 계산합니다.
 - **`retrieval=true` 모드 (v0.12.0):** GraphRAG 레이어의 검색 순수 성능(Table/Column Recall@k, Join Path Recall, Value Evidence Recall 및 graph-vs-plain recall gain)만을 분리 추출하여 정밀 측정합니다.
 - `profile` 이 지정된 경우, 기대 SQL을 대상 DB(postgres/mysql/mariadb)에서 read-only 실행하여 실행 성공률과 반환 행 수의 타당성을 검사하고 최종 **정밀 지표 리포트**를 출력합니다.
- **비즈니스 목적:** 데이터셋 스키마 정보나 카탈로그 룰 변경 시, 기존 시스템의 SQL 변환 정확도가 후퇴 (Regression)했는지 검증하는 품질 게이트입니다.

23) `suggest_joins`

- **입력 규격 (Parameters):** `tables` (array, 선택), `top_k` (integer, 선택)
- **내부 동작 알고리즘 (`joinsuggest.go`):**
 - 카탈로그 내 테이블 명세를 기반으로, 단일 컬럼 PK 속성명을 가진 마스터 테이블이 존재하나 조인 관계(`relations`)에 누락되어 고립된 잠재 엣지를 발굴하여 `overrides.json` 스니펫을 추천함

니다.

- **비즈니스 목적:** 스키마 전개 시 누락된 조인 경로를 기계적 탐색을 기반으로 찾아내어 카탈로그 지식을 확충하는 유지보수 가이드 도구입니다.

24) `list_datasets`

- **입력 규격 (Parameters):** 없음
- **내부 동작 알고리즘 (`datasets.go`):**
 - 서버 카탈로그 엔진에 등록된 18종의 데이터셋 메타 명세(glossary, metrics, overrides 등) 레지스트리 상태를 전수 확인합니다.
 - 각 데이터셋의 물리 파일 경로, 데이터 건수, 적재 용량 및 기동 컴파일 과정 중 발생했던 개별 LoadIssue 건수를 매핑하여 출력합니다.
- **비즈니스 목적:** 운영자가 데이터셋 원장의 세부 물리 보관 상태 및 라이프사이클을 통제하는 원장을 제공합니다. (메타 DB 모드에서는 Postgres `jamypg_datasets` 테이블이 원천 저장소로 동작합니다.)

25) `get_dataset`

- **입력 규격 (Parameters):** `name` (string, 필수), `sample_rows` (integer, 선택)
- **내부 동작 알고리즘 (`datasets.go`):**
 - `list_datasets` 에서 조회된 특정 데이터셋의 메타 설명 정보를 취득하고, 타깃 JSON 파일의 원본을 로드하여 `sample_rows` 수량(기본 5개, 최대 50개)만큼의 JSON 원시 샘플 레코드를 리턴합니다.
- **비즈니스 목적:** 데이터셋 전체 파일을 한 번에 불러오지 않고, 실시간 운영 콘솔이나 터미널 상에서 메타 데이터 일부를 안전하게 부분 모니터링하기 위해 사용합니다.

26) `put_dataset`

- **입력 규격 (Parameters):** `name` (string, 필수), `content` (json-data, 필수), `force` (boolean, 선택)
- **내부 동작 알고리즘 (`datasets.go` , `server.go`):**
 - 수정 요청이 유입되면 `dataMu` 뮤텍스를 점유하고 원본 파일을 백업한 뒤 가상 버퍼 공간에 신규 JSON 데이터를 저장하고 컴파일 유효성을 체크하여 원자적으로 핫스왑 혹은 롤백 처리합니다.
- **비즈니스 목적:** 서비스 무중단 상태에서 안전하게 금융 카탈로그 정보를 패치하고 오류 시 실시간 복원하는 거버넌스 핫스왑 도구입니다.

27) `remove_dataset`

- **입력 규격 (Parameters):** `name` (string, 필수)
- **내부 동작 알고리즘 (`datasets.go` , `server.go`):**
 - 해당 데이터셋 명세를 레지스트리에서 스캔하여 필수 여부를 판단하고 선택형 데이터셋인 경우에만 물리 파일을 삭제(백업 파일 생성 후) 처리하며 재컴파일 및 핫스왑을 수행합니다.
- **비즈니스 목적:** 선택형 메타 지식 정보를 카탈로그 상에서 완전히 제거하고 메모리를 핫스왑 정리하는 자원 정리기입니다.

28) `reload_catalog`

- **입력 규격 (Parameters):** 없음
- **내부 동작 알고리즘 (`server.go`):**
 - 디스크 상의 전체 JSON 18종 데이터 셋을 처음부터 재컴파일하여 `Catalog` 구조체 주소를 메모리 상에 안전하게 스위칭합니다.
- **비즈니스 목적:** 디스크 상태와 인메모리 카탈로그 주소 동기화를 수동 강제하는 유지보수용 도구입니다.

5. AI 인프라실 분석 및 종합 의견

5.1. 금융 AI 거버넌스 관점의 효용성

JAMYPG MCP 서버는 모든 데이터 접근 권한을 카탈로그 메모리 내에서 1차 통제하고, 33종의 룰 엔진 (`validate_sql`) 및 실측 플랜 검증(`explain_sql`)을 통과한 무결한 읽기 전용 질의만 대상 DB(PostgreSQL·MySQL·MariaDB) 세션에 전달하는 **다층 보안 아키텍처**를 가지고 있습니다. 여기에 `internal/dbconn` 커넥터가 DSN 수준 read-only 세션 강제와 방언별 위험 함수 차단(`pg_sleep`, `LOAD_FILE`, `BENCHMARK` 등)을 추가로 가동합니다. 특히 v0.12.0부터 도입된 **GraphRAG 기반 `retrieve_context`** 기법은 기존 렉시컬 검색이 지닌 단순 단어 매치 한계를 극복하고, 복잡한 다중 관계 토폴로지를 1-hop 확장으로 커버하여 누락 없는 안전한 쿼리 골격을 조립하는데 핵심적인 기여를 하고 있습니다.

5.2. 인프라 운영 관리 지침

- **조인 갭 분석 연계:** `suggest_join_relations` 분석 도구를 주간 단위 스케줄러로 기동하여, 골든셋 검증 갭을 정기 식별하고 `relations.json` 을 갱신함으로써 쿼리 성공 정확도를 상시 점검해야 합니다

다.

- **재질문 이력 피드백 학습 관리:** `prepare_sql_context` 에 사용자 재질문 해소 이력이 자동 축적되므로, 현업의 의도 왜곡으로 엉뚱한 테이블에 usage 가중치가 쏠리지 않도록 피드백 outcome 로그를 어드민 포털에서 주기적으로 관리해야 합니다.